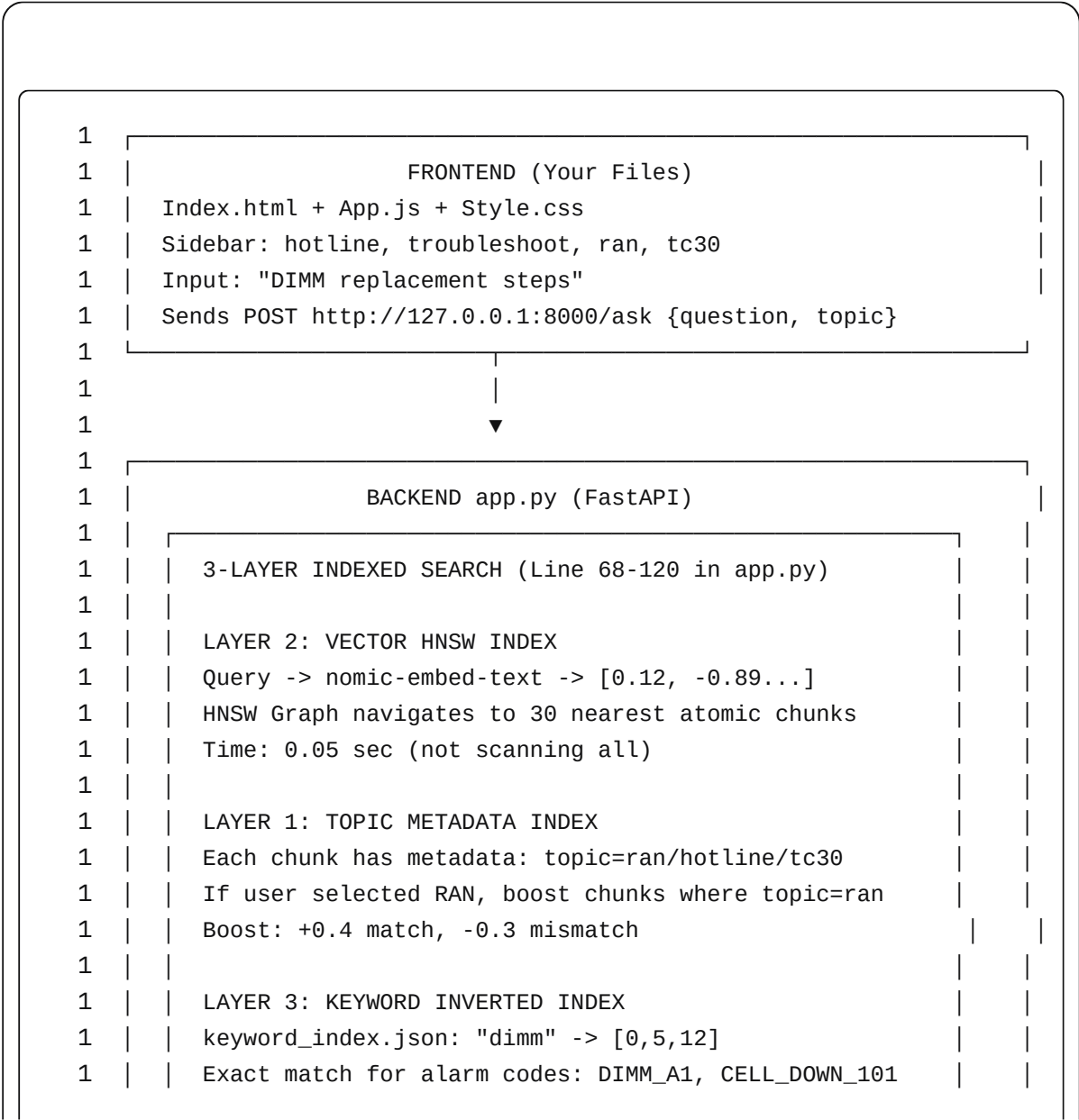


TASS - TMO AI Support System - Local RAG Backend

100% Local | Ollama | Atomic Chunking | 3-Layer Indexing

Backend for your frontend: `Index.html + App.js + Style.css`
Supports: PDF with images, DOCX with images, TXT

1. Architecture - How Indexing Works



```

1 | | Boost: keyword_count + fuzzy_score | |
1 | | | |
1 | | FINAL SCORE = 50% vector + 25% keyword + 15% fuzzy + | |
1 | | topic_boost | |
1 | | Top 6 atomic facts selected | |
1 | | | |
1 | | | |
1 | | | |
1 | | | |
1 | | LLM Generation (Ollama llama3.2) | |
1 | | Prompt: Atomic Context + Question + Topic | |
1 | | Output: HTML with <b>, <ul>, <table>, <img> tags | |
1 | | | |
1 | | | |
1 | | Returns: {answer: HTML, sources: [{file, page}], images: [/images/ | |
1 | | | |
1 | | | |
1 | | | |
1 | | | |
1 | | FRONTEND App.js Renders | |
1 | | - Shows answer as HTML | |
1 | | - Shows images from http://127.0.0.1:8000/images/xxx.jpg | |
1 | | - Shows Sources: hotline.pdf page 5 | |
1 | | - Download as .html button | |
1 | | | |
1 | | | |
1 | BUILD TIME (ingest.py):
1 | data/ (hotline/, ran/, tc30/)
1 | -> extract_pdf_with_images() [IMAGE INDEX]
1 | -> detect_topic_from_path() [TOPIC INDEX]
1 | -> atomic_chunking() with llama3.2 [ATOMIC INDEX]
1 | -> build_keyword_index() -> keyword_index.json [KEYWORD INDEX]
1 | -> Chroma.from_documents() -> vector_store/ [VECTOR HNSW + METADATA

```

2. Index Types Explained

Index	Where Added	File	How it Works
Topic Metadata Index	<code>detect_topic_from_path()</code> Line 24 in ingest.py, <code>metadata["topic"]</code> Line 158	<code>ingest.py</code> + <code>app.py</code> Line 85	Folder name -> topic tag. Chroma B-Tree filters <code>topic==ran</code> instantly.
Vector HNSW Index	<code>Chroma.from_documents()</code> Line 217 in ingest.py	<code>vector_store/</code> folder	Text -> 768 numbers -> HNSW graph road map for semantic search
Keyword Inverted Index	<code>build_keyword_index()</code> Line 183 in ingest.py	<code>keyword_index.json</code>	Word -> [chunk IDs]. Exact match for DIMM_A1, CELL_DOWN codes
Image Index	<code>page.get_images()</code> Line 53 in ingest.py, <code>app.mount("/images")</code> Line 26 in app.py	<code>extracted_images/</code> folder	Saves images from PDF/DOCX, links to atomic chunk via metadata
Atomic Chunking Index	<code>atomic_chunking()</code> function Line 115 in ingest.py	-	One procedure = One chunk, not 600 chars. Uses Ollama to split by meaning

3. Project Structure

```

1 tass-backend/
1 |─ data/
1 |   │─ hotline/          # Put FOPS, DIMM docs here -> topic=hotline
1 |   │   └─ fops.pdf
1 |   │─ ran/              # RAN docs -> topic=ran
1 |   │   └─ cell_down.docx
1 |   │─ tc30/             # TC30 docs -> topic=tc30
1 |   └─ troubleshoot/     # TSH docs -> topic=troubleshoot
1 └─ extracted_images/     # Auto-created - all diagrams from PDFs
1 └─ vector_store/         # Auto-created - ChromaDB with 3 indexes
1 └─ keyword_index.json    # Auto-created - keyword inverted index
1 └─ ingest.py             # Builds all indexes - run once per doc upda
1 └─ app.py                # FastAPI server - serves frontend
1 └─ requirements.txt      # Dependencies
1 └─ README.md             # This file

```

4. Step-by-Step Setup (Beginner)

Step 0: Install Ollama

Download from <https://ollama.com> - Install - Then in terminal:

bash

```

1 ollama pull llama3.2
1 ollama pull nomic-embed-text
1 ollama serve
1 # Keep this terminal running

```

Step 1: Python Environment

bash

```

1 # Create folder

```

```
1 mkdir tass-backend
1 cd tass-backend
1
1 # Create venv
1 python -m venv venv
1 # Windows: venv\Scripts\activate
1 # Mac/Linux: source venv/bin/activate
1
1 pip install -r requirements.txt
```

Step 2: Add Documents

Organize by topic for best indexing:

bash

```
1 mkdir -p data/hotline data/ran data/tc30 data/troubleshoot
1 # Copy your PDFs/DOCXs into respective folders
```

Step 3: Build Indexes

bash

```
1 python ingest.py
```

Expected output:

```
1 [EXTRACT] Reading PDF: data/ran/cell_down.pdf
1 -> Extracted 12 pages with 5 images
1 [INDEXING] Starting ATOMIC CHUNKING...
1 [INDEXING] Created 89 ATOMIC chunks
```

```
1 [INDEXING] Building KEYWORD INVERTED INDEX...
1   -> Keyword index built: 1243 unique keywords
1 [INDEXING] Building VECTOR + METADATA INDEX in ChromaDB...
1 SUCCESS! All indexes built:
1   1. TOPIC METADATA INDEX
1   2. VECTOR HNSW INDEX
1   3. KEYWORD INVERTED INDEX
1   4. IMAGE INDEX
```

Step 4: Start Backend

bash

```
1 uvicorn app:app --reload --port 8000
```

Open <http://127.0.0.1:8000> in browser - should show indexes status.

Step 5: Start Frontend

Double-click `Index.html` from your original folder OR copy `Index.html`, `App.js`, `Style.css` into `tass-backend` and open.

Type query: `DIMM replacement steps` - Should return answer with images.

5. API Details - For Your App.js

Endpoint: `POST http://127.0.0.1:8000/ask`

Request:

json

```
1 {
1   "question": "DIMM replacement steps",
1   "topic": "hotline"
```

```
1 }
```

Response:

json

```
1 {
1   "answer": "<b>DIMM Replacement:</b><br>1. Check LED...<br><img src=
1   "sources": [{"file": "fops.pdf", "page": 5, "topic": "hotline"}],
1   "images": ["/images/fops_p5_img0.jpg"]
1 }
```

Your `App.js` already handles this format (Line 246-273).

Fallback endpoint: `POST /query` - same as `/ask`

6. Adding New Documents - Incremental

Instead of deleting DB every time (current code does for simplicity), for production use:

python

```
1 # In ingest.py, replace delete logic with:
1 vector_db = Chroma(persist_directory=DB_PATH, embedding_function=embe
1 vector_db.add_documents(new_atomic_docs)
```

7. Troubleshooting

- `Connection refused` -> Ollama not running. Run `ollama serve` in separate terminal.
- `No files found` -> Check `data/` has PDFs. Use subfolders like `data/ran/`.

- Frontend shows `Failed: ...` -> Backend not running on 8000. Check `uvicorn app:app...` terminal.
- Images not showing -> Check `extracted_images/` has files and backend mounted `/images`. Your App.js Line 263 converts `/images/` to `http://127.0.0.1:8000/images/`.

8. Why Atomic Chunking Not Size Chunking?

Size chunking (bad for your case):

Chunk 1: "DIMM failure: Step 1 Check LED, Step 2 Power off..."

Chunk 2: "Step 3 Remove DIMM, Step 4 Replace..."

Bot returns only Step 1-2 - incomplete procedure.

Atomic chunking (your requirement):

Chunk 1: "DIMM failure complete procedure: Step 1 Check LED, Step 2 Power off, Step 3 Remove DIMM, Step 4 Replace, Step 5 Verify"

= One atomic fact = One complete answer.

Built using Ollama LLM with delimiter `|||END|||`.

Next: Add more topics? Just create folder `data/audit/` -> auto-detects topic=Audit.